

Firmware Deployment Guide

Flashing Cross-Architecture Binaries: Dual-Core ESP32 & ATmega328P Supervisor

IMPORTANT POWER ISOLATION MANDATE

Before connecting either microcontroller to your Mac via USB, ensure the high-voltage 12.6V XT60 LiPo battery is completely disconnected from the power distribution board. Flashing devices while cross-connected to an active high-voltage circuit introduces voltage spikes that can permanently damage both microcontrollers and your host computer's USB sub-systems.

Section 1 — Core 1 & Core 0: ESP32 Firmware Deployment

Your 250 Hz flight loop controller is written for native execution inside the ESP-IDF framework environment. Because it interacts directly with custom register allocations, compiling and flashing should be done directly using the command-line interface tools.

1.1 Interface Environment Setup

1. Open your Mac terminal workspace and navigate to your flight controller root directory.
2. Initialize your local environment path references by sourcing the active ESP-IDF toolchain exports file:

```
. $HOME/esp/esp-idf/export.sh
```

3. Identify your target USB device path. Plug the ESP32 into your Mac and list operational physical device entries using your terminal:

```
ls /dev/cu.usbserial*
```

Record this path string (e.g., /dev/cu.usbserial-1410).

1.2 Compilation & Flashing Sequence

1. **Set Target Device:** Ensure the project configuration is strictly locked to the ESP32 silicon base archetype:

```
idf.py set-target esp32
```

2. **Open Visual Config (Optional Verification):** If checking CPU core allocations, clock speeds, or flash sizes, load the internal menu configuration interface:

```
idf.py menuconfig
```

3. **Compile Binaries:** Execute a structural project build to compile your code blocks into cross-compiled image artifacts:

```
idf.py build
```

4. **Deploy and Flash:** Execute the structural upload command targeting your device path. Substitute your recorded serial connection identity string here:

```
idf.py -p /dev/cu.usbserial-1410 flash
```

5. **Monitor Console Real-Time:** Instantly check your print statements, Wi-Fi initialization hooks, and task status outputs by attaching the serial monitor interface:

```
idf.py -p /dev/cu.usbserial-1410 monitor
```

To break away from this live log environment, utilize the hotkey macro: Ctrl +].

Section 2 — Coprocessor: ATmega328P / Arduino Deployment

The secondary ATmega328P coprocessor manages real-time system monitoring, ADC battery voltage division calculations, and safety alarms. You can upload its supervisor firmware via your Mac terminal by calling the background AVR compiler directly.

2.1 Automated Toolchain Execution (CLI Method)

1. Install the fundamental development and flashing utilities via Homebrew if they are missing from your environment:

```
brew install avr-gcc avrdude
```

2. Identify the secondary physical USB serial path mapping corresponding to your connected Arduino board:

```
ls /dev/cu.wchusbserial* # Standard for CH340 chip clones
ls /dev/cu.usbmodem* # Standard for official Arduino hardware
```

3. Compile your local C file artifact directly down into an architecture-native executable production image file:

```
avr-gcc -F_CPU=16000000UL -mmcu=atmega328p -Wall -Os -o supervisor.elf supervisor.c
```

4. Convert the compiled ELF file container structure down into an isolated, pure Intel-HEX file format stream:

```
avr-objcopy -O ihex -R .eeprom supervisor.elf supervisor.hex
```

5. Execute the low-level flashing instruction command to upload the hex binary over the internal serial bootloader structure:

```
avrdude -F -V -c arduino -p m328p -P /dev/cu.wchusbserial11420 -b 115200 -U  
flash:w:supervisor.hex:i
```

2.2 Alternative Integrated Development Environment (IDE) Method

1. Launch your local installation of the Arduino IDE program.
2. Copy your coprocessor code implementation text block cleanly into the master code editor interface window.
3. Navigate to the top dropdown taskbar menus and set the definitions under: **Tools** → **Board** → **Arduino AVR Boards** → **Arduino Nano** (or Uno).
4. If utilizing a common third-party development clone board, shift your configuration definition selection to: **Tools** → **Processor** → **ATmega328P (Old Bootloader)**.
5. Select your matching hardware serial link pathway under **Tools** → **Port**.
6. Click the structural **Upload arrow icon** located in the upper left header panel to push the compiled supervisor binary to your hardware.

Section 3 — Deployment Integrity & Verification Matrix

Once both target architectures are fully flashed and disconnected from your host machine, perform this sequential verification checklist to confirm operational readiness:

Step Number	Action Requirement	Expected Visual/Functional Response Feedback
1	Connect your Mac USB link line directly back to the ESP32 module.	Onboard status LED illuminates. Serial log outputs initialization confirmations.
2	Attach a standard USB power bank line directly to the ATmega328P socket.	Status LED illuminates. If the main battery remains isolated, the alert piezo buzzer should chirp to notify low voltage.
3	Re-engage the 5V UBEC line, reinstate the shared UART links, and plug in the 12.6V LiPo battery.	Both boards boot simultaneously without external aid. The alert buzzer immediately becomes silent once the ADC registers detect nominal battery voltage.